

TECHNICAL SPECIFICATION & UX WRITEUP

Spark Estimator PWA — Submission Deliverable

1. Most Interesting Design / UX Decision

Isolated Indented Cards with Card-Wide Tap Hits: Walking active construction sites requires one-handed, high-speed mobile inputs. Standard forms requiring users to tap small, exact circular check boxes are slow and error-prone in the field. We expanded the touch hit zone to the entire top row of each card (the name, cost label, and grey background). Once selected, the input sub-elements (quantities, notes, photos, and voice notes) expand under a strict 40px left-indentation block. This separates edit actions from toggle actions, preventing accidental toggling/deselection on mobile screens.

2. What's Broken or Fragile

Signal-Zero First Bootstrap: Excel spreadsheet construction and photo folder ZIP packaging depend on third-party CDN libraries (SheetJS/xlsx-js-style and JSZip). Although the Service Worker caches these files automatically for robust offline execution, if an estimator logs in for the very first time in a zero-signal concrete basement without prior internet connection, the export functionality degrades. It warns the user to establish a brief signal connection to pull library caches on first boot.

3. Creative Addition: Real Estate Deal Analyzer

On-Site Profit Analysis: Estimating repair budgets is useless without comparing them against the purchase price and ARV (After Repair Value). We built an interactive SVG allocation chart that calculates the financial deal directly alongside repair walkthroughs. It computes closing costs, holding costs, interest financing rates, and contingencies to output net profit, ROI, and profit margins. This turns the app from a simple field worksheet into an indispensable deal-closing sales tool for agents.

4. Next Shipments (With 2 More Days)

100% Offline Autonomy & Computer Vision: First, we would inline the base64-compiled SheetJS and JSZip assets directly inside the main document. This removes CDN network dependencies on first launch. Second, we would deploy a client-side camera object-detection model (TensorFlow.js) so agents can point their lens at water damage or cracked drywall to auto-fill line items from the catalog.

5. The Role of Generative AI

Test-Driven Development & Validation: AI acted as a pair programmer: verifying CSV line parsing edge cases, writing IndexedDB/Dexie fallback handlers, and generating an embedded test runner (`window.runTests()`) to validate arithmetic routines. AI also helped ensure PWA caching strategies complied with strict offline guidelines.